

How does Merge Sort work?

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the divide-and-conquer approach to sort a given array of elements.

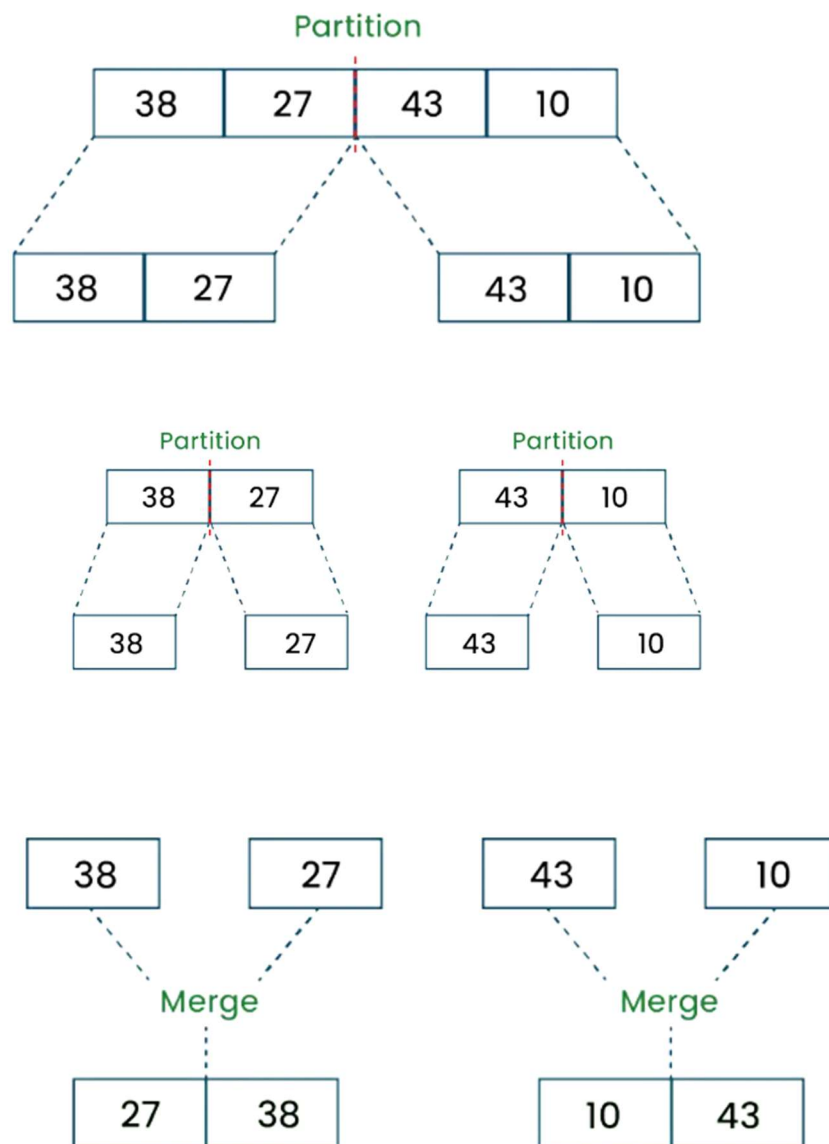
Here's a step-by-step explanation of how merge sort works:

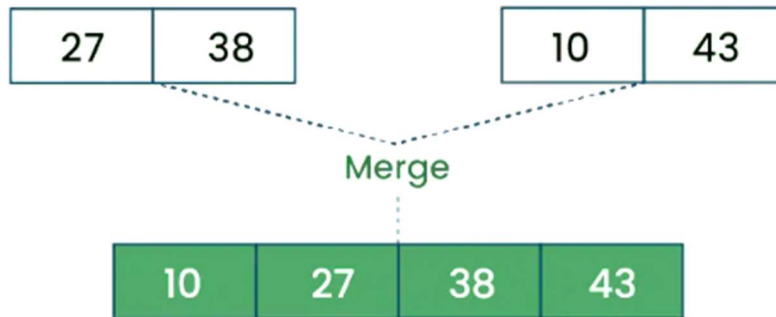
Divide: Divide the list or array recursively into two halves until it can no more be divided.

Conquer: Each subarray is sorted individually using the merge sort algorithm.

Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Let's sort the array or list [38, 27, 43, 10] using Merge Sort





Let's look at the working of above example:

Divide:

[38, 27, 43, 10] is divided into [38, 27] and [43, 10].

[38, 27] is divided into [38] and [27].

[43, 10] is divided into [43] and [10].

Conquer & Merge:

Merge [38] and [27] to get [27, 38].

Merge [43] and [10] to get [10, 43].

Merge [27, 38] and [10, 43] to get the final sorted list [10, 27, 38, 43]

Therefore, the sorted list is [10, 27, 38, 43].

Quick Sort

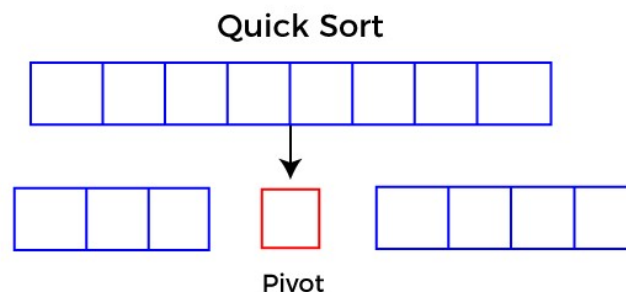
Quick sort is the widely used sorting algorithm that makes comparisons in average case for sorting an array of n elements. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquers approach.

Divide and conquer is a technique of breaking down the algorithms into sub problems, then solving the sub problems, and combining the results back together to solve the original problem.

- **Divide:** In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.
- **Conquer:** Recursively, sort two subarrays with Quick sort.
- **Combine:** Combine the already sorted array.

Quick sort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.

After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.



Choosing the pivot:

Some of the ways of choosing a pivot are as follows -

- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element or the leftmost element of the given array.
- Select median as the pivot element.

Algorithm:

```
QUICKSORT (array A, start, end)
{
    if (start < end)
    {
        p = partition(A, start, end)
        QUICKSORT (A, start, p - 1)
        QUICKSORT (A, p + 1, end)
    }
}
```

Working of Quick Sort Algorithm

To understand the working of quick sort, let's take an unsorted array. It will make the concept more clear and understandable.

24	9	29	14	19	27
----	---	----	----	----	----

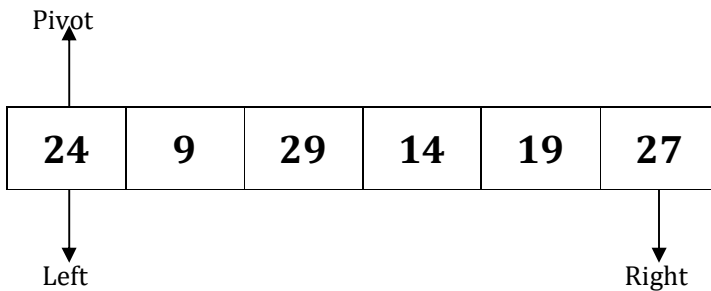
In the given array, we consider the leftmost element as pivot. So, in this case,

$a[\text{left}] = 24$

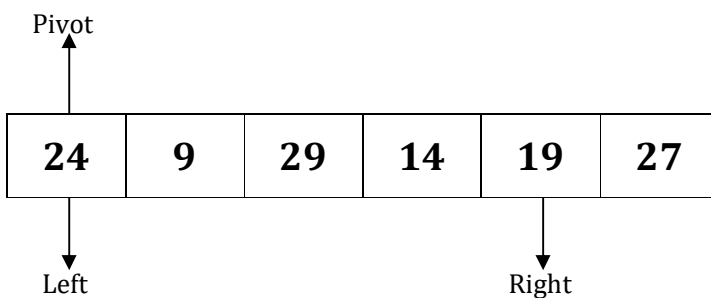
$a[\text{right}] = 27$

$a[\text{pivot}] = 24$

Since, pivot is at left, so algorithm starts from right and move towards left.

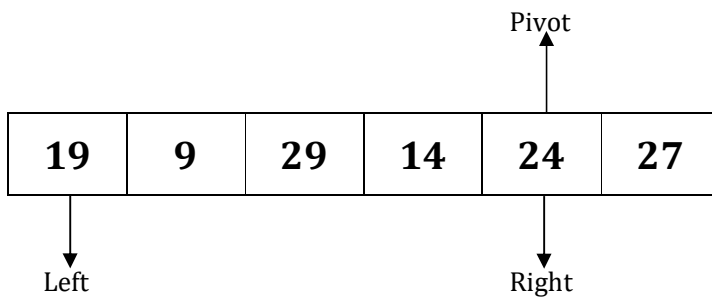


Now, $a[\text{pivot}] < a[\text{right}]$, so algorithm moves forward one position towards left, i.e. –



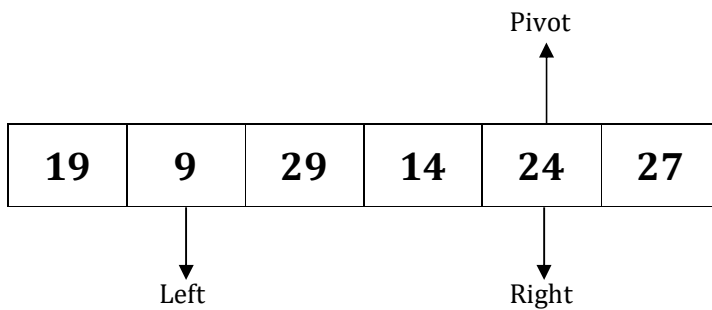
Now, $a[\text{left}] = 24$, $a[\text{right}] = 19$, and $a[\text{pivot}] = 24$.

Because, $a[\text{pivot}] > a[\text{right}]$, so, algorithm will swap $a[\text{pivot}]$ with $a[\text{right}]$, and pivot moves to right, as -

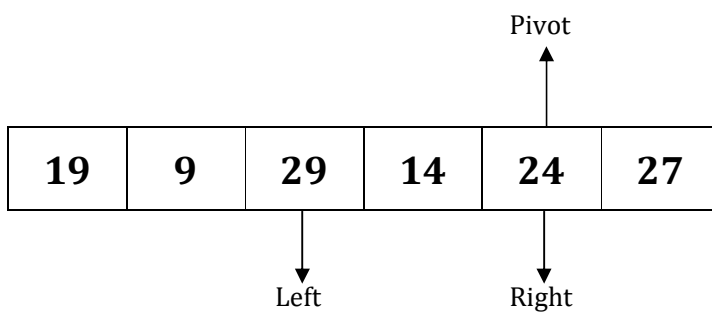


Now, $a[\text{left}] = 19$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. Since, pivot is at right, so algorithm starts from left and moves to right.

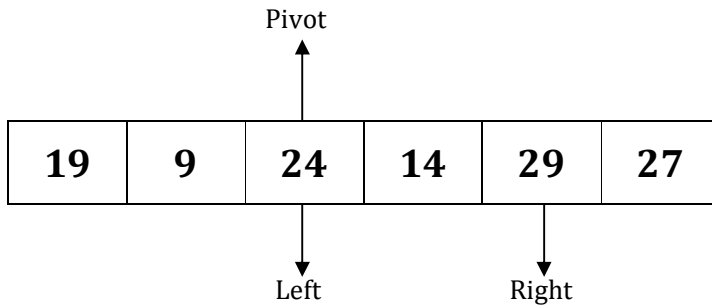
As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -



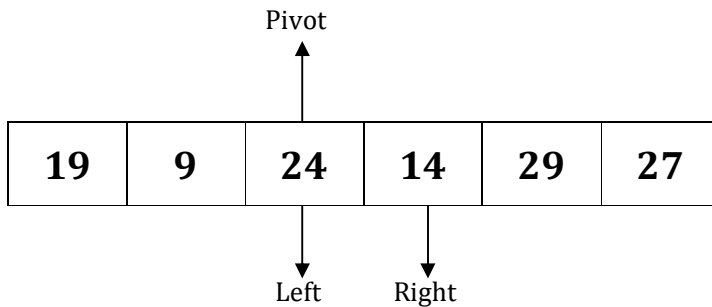
Now, $a[\text{left}] = 9$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -



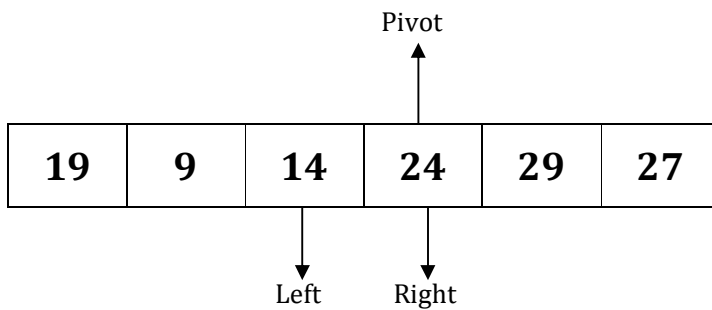
Now, $a[\text{left}] = 29$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{left}]$, so, swap $a[\text{pivot}]$ and $a[\text{left}]$, now pivot is at left, i.e. -



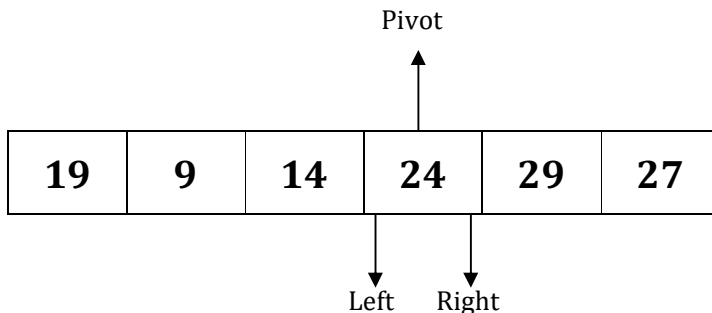
Since, pivot is at left, so algorithm starts from right, and move to left. Now, $a[\text{left}] = 24$, $a[\text{right}] = 29$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{right}]$, so algorithm moves one position to left, as –



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 14$. As $a[\text{pivot}] > a[\text{right}]$, so, swap $a[\text{pivot}]$ and $a[\text{right}]$, now pivot is at right, i.e. –



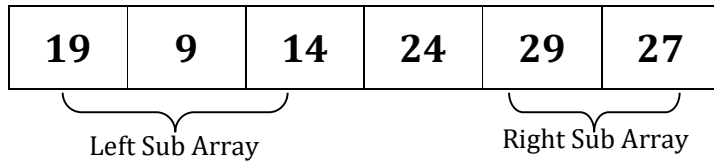
Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 14$, and $a[\text{right}] = 24$. Pivot is at right, so the algorithm starts from left and move to right.



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 24$. So, pivot, left and right are pointing the same element. It represents the termination of procedure.

Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.



Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be –

9	14	19	24	27	29
----------	-----------	-----------	-----------	-----------	-----------